

LANGCHAIN AJAN GELİŞTİRME ÇALIŞMA RAPORU

LLM Akıl Yürütme Motorları ve Özelleştirilmiş Ajan Yapıları

1. Giriş ve Büyük Dil Modellerinin Yeni Tanımı

Geleneksel olarak, Büyük Dil Modelleri (LLM'ler) genellikle geniş bir internet verisi üzerinde eğitilmiş sabit bir **bilgi deposu (knowledge store)** olarak görülür. Bu bakış açısına göre LLM, kendisine yöneltilen sorulara hafızasındaki bilgileri çağırarak yanıt veren statik bir ansiklopedi gibidir. Ancak, LangChain çerçevesinde LLM'leri konumlandırmak için çok daha güçlü bir yaklaşım mevcuttur: LLM'leri birer **akıl yürütme motoru (reasoning engine)** olarak kullanmak.

Gündelik Hayattan Örnek (Analoji):

Geleneksel LLM yaklaşımı, her tıbbi kitabı kelimesi kelimesine ezberlemiş ancak hastane dışında hiçbir araçla bağlantısı olmayan izole bir doktora benzer. Yeni yaklaşım olan Akıl Yürütme Motoru ise bu doktorun önüne laboratuvar test kitleri, MR tarayıcıları ve hasta geçmiş dosyalarını koyup ona bu araçları ne zaman kullanacağını seçme yetkisi vermek gibidir. Doktor her hastalığı ezbere bilmek zorunda kalmaz; doğru sonuca ulaşmak için hangi aracı (tool) çağıracağını akıl yürüterek belirler.

2. LangChain Ajanları (Agents) ve ReAct Metodolojisi

LangChain'in **Ajanlar (Agents)** çerçevesi, dil modelini akıl yürütme motoru olarak konumlandırıp, ona hangi eylemleri hangi sırayla gerçekleştireceğine karar verme yeteneği tanır. Ajanlar, dile getirilmiş bir problem karşısında adım adım planlama yapar, gerekli araçları belirler, çalıştırır ve çıkan sonuçları (gözlemleri) değerlendirerek nihai cevaba ulaşır. Bu süreç, günümüzde son derece popüler olan ve LangChain ajanlarında standart olarak kullanılan **ReAct (Reason + Act)** metodolojisine dayanır.

ReAct Nedir?

ReAct, dil modelinin daha iyi sonuçlar üretebilmesi için **düşünce (Thought)** ve **eylem (Action)** süreçlerini birleştiren bir yönlendirme (prompting) stratejisidir. Model, bir soruyu doğrudan yanıtlamak yerine sırasıyla şu adımları izler:

- Thought (Düşünce):** Problemi analiz eder ve ne yapması gerektiğine karar verir.
- Action (Eylem):** Belirlediği bir aracı (örn. Wikipedia veya Arama Motoru) çağırır.
- Observation (Gözlem):** Aracın döndürdüğü sonucu okur ve süreci tekrarlayarak nihai cevaba (Final Answer) ulaşır.

3. Uygulamalı Kod Örnekleri: Hazır Araçlarla Ajan Kurulumu

LangChain üzerinde hazır araçlar (DuckDuckGo Arama Motoru ve Wikipedia) entegre edilmiş bir ajanın kurulumu oldukça basittir. Ajan, eğitim verilerinde yer almayan güncel bilgileri (örneğin 2022 Dünya Kupası kazananı) internet aramasıyla bulur veya Wikipedia üzerinden tarihsel verileri sorgular.

Gerekli Kütüphanelerin Kurulumu

Ajanımızın internette arama yapabilmesi ve Wikipedia'ya erişebilmesi için aşağıdaki Python kütüphanelerini kurması gerekir:

```
pip install duckduckgo-search wikipedia
```

Ajanın Başlatılması ve Yapılandırılması

Aşağıdaki kod örneğinde, modelin çıktıları parse ederken oluşabilecek hataları yönetmek üzere **handle_parsing_errors=True** parametresi verilmiştir. Çıktı ayrıştırıcıları (Output Parsers), dil modelinin metinsel çıktıları doğrudan yürütülebilir eylemlere ve eylem girdilerine (Action Input) dönüştüren kritik bileşenlerdir.

```
from langchain.agents import load_tools, initialize_agent
from langchain.agents import AgentType
from langchain.chat_models import ChatOpenAI

# 1. Dil modelini (akıl yürütme motorunu) tanımla
llm = ChatOpenAI(temperature=0.0)

# 2. Hazır LangChain araçlarını yükle (DuckDuckGo Arama ve Wikipedia)
tools = load_tools(["ddg-search", "wikipedia"])

# 3. Ajanı başlat
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.CHAT_ZERO_SHOT_REACT_DESCRIPTION,
    handle_parsing_errors=True
)
```

Çalışma Senaryosu 1: Güncel Olay Sorgulama

Modelin eğitim verilerinin kesim tarihi (yaklaşık 2021) sonrası gerçekleşen bir olay olan '2022 Dünya Kupası'nı kazanan ülke' sorulduğunda ajan şu adımları izler:

```
# Ajanı çalıştırıyoruz
agent.run("2022 Dünya Kupasını kim kazandı?")

# ARKA PLAN ÇALIŞMA SÜRECİ (TRACE):
# Thought: Bu güncel bir bilgi. DuckDuckGo Search aracını kullanmalıyım.
# Action: duckduckgo_search
# Action Input: "2022 World Cup winner"
# Observation: [İnternette gelen arama sonuçları verisi]
# Thought: Arama sonuçlarına göre Arjantin kupayı kazandı.
# Final Answer: 2022 Dünya Kupasını Arjantin kazanmıştır.
```

4. Özelleştirilmiş Araç (Custom Tool) Tasarımı

LangChain'in en güçlü yanlarından biri, ajanlarınızı tamamen kendinize ait özel veri tabanlarına, API'lere ve fonksiyonlara bağlayabilmenizdir. Bunun için yapılması gereken tek şey, sıradan bir Python fonksiyonuna **@tool** dekoratörünü eklemektir.

Docstring'in Kritik Rolü

Özel bir araç yazarken fonksiyonun içine yazılan **docstring (açıklama metni)** son derece önemlidir. Çünkü LangChain, bu açıklama metnini doğrudan dil modeline (LLM) gönderir. Model, hangi aracın ne zaman ve hangi kurallarla çağrılacağını bu docstring'e bakarak anlar. Örneğin, girdi olarak boş bir dize alması gerektiğini modele bu şekilde bildiririz.

```
from langchain.agents import tool
from datetime import datetime

@tool
def time(text: str) -> str:
    """Bugünün tarihini döndürür.
    Bu aracı çağırarak için girdi her zaman boş bir dize (') olmalıdır.
    """
    return str(datetime.now().date())

# Yeni aracı ajanımızın araç listesine ekliyoruz
tools = tools + [time]

# Ajanı yeniden başlatıyoruz
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.CHAT_ZERO_SHOT_REACT_DESCRIPTION,
    handle_parsing_errors=True
)
```

Çalışma Senaryosu 2: Özel Aracı Kullanma

Ajanımıza bugünün tarihini sorduğumuzda, dil modeli yazdığımız açıklama metnini okur ve eylemi doğru şekilde tetikler:

```
agent.run("Bugünün tarihi nedir?")

# ARKA PLAN ÇALIŞMA SÜRECİ (TRACE):
# Thought: Bugünün tarihini bilmek için 'time' aracını çağırmalıyım. Girdi boş dize olmalı.
# Action: time
# Action Input: ""
# Observation: 2023-05-21
# Thought: 'time' aracı bugünün tarihini 2023-05-21 olarak döndürdü.
# Final Answer: Bugünün tarihi 2023-05-21'dir.
```

5. Sonuç ve Özet Değerlendirme

Özellik	Bilgi Deposu (Knowledge Store)	Akıl Yürütme Motoru (Reasoning Engine)
Çalışma Mantığı	Bilgiyi statik bellekten doğrudan çağırır.	Bilgiyi okur, plan yapar ve araçları kullanarak sonuca ulaşır.
Güncellik	Modelin eğitildiği tarihten sonrasını bilmez.	Arama motoru vb. araçlarla her zaman en güncel bilgiye erişir.
Genişletilebilirlik	Sabittir, modeli yeniden eğitmek gerekir.	Yeni API ve fonksiyonlar (@tool) eklenerek sınırsızca genişletilir.